

School of Electronic Engineering and Computer Science

Queen Mary University of London

MSc PROJECT DEFINITION 2025–26

Project Title:	Coordinating Speculative Agent Workloads Over Data Backends: Middleware Policies for Redundancy Elimination
Supervisor:	[Supervisor Name]
Student Name:	Pash Gorenkin
Student E-mail:	gorenkinp@gmail.com

This project aims to design, implement, and evaluate a middleware layer that coordinates multiple LLM-based agents working in parallel against shared database backends. The core research challenge is determining how to optimally share information between speculative agents to reduce redundant database queries without degrading answer quality.

Specifically, the project addresses the problem that when multiple agents are launched in parallel on the same text-to-SQL task, they independently explore the database schema, issue overlapping queries, and repeat identical work. The middleware intercepts these interactions and applies coordination policies ranging from passive caching to phase-aware broadcasting, characterising the trade-off space between coordination overhead and redundancy elimination.

The research question is: What is the most effective architecture for coordinating speculative agent workloads over data backends, and where are the trade-offs between coordination overhead and redundancy elimination?

- Establish redundancy baselines by running 3, 10, and 25 parallel agents on the BIRD text-to-SQL benchmark and measuring query overlap, sub-expression duplication, and time-to-solution.
- Design and implement a four-component middleware architecture: query interceptor with AST-level SQL normalisation (via sqlglot), shared discovery board, coordination policy engine, and monitoring/metrics layer.
- Implement and evaluate five coordination policies: no coordination (P0), passive caching (P1), active result sharing (P2), discovery broadcasting (P3), and staged/phase-aware coordination (P4).
- Conduct a full evaluation matrix across all policies, agent counts (3, 10, 25), and at least two LLM backends, measuring seven metrics: total queries, cache hit rate, database load, wall-clock time, execution accuracy, token cost, and coordination overhead.
- Characterise the trade-off space with statistical rigour (95% bootstrap confidence intervals), identifying when coordination helps, when it hurts, and why.

Alignment with programme learning outcomes:

- Demonstrates independent research methodology through experimental design and statistical evaluation.
- Applies distributed systems principles (caching, coordination, consistency models) to a novel domain (LLM agent orchestration).
- Produces a working software system evaluated against established benchmarks with reproducible results.

Phase 1 — Infrastructure & Baselines (Month 1). Set up DuckDB with the BIRD dataset. Build an async Python orchestrator to run multiple LLM agents in parallel with full query logging. Run baseline experiments with 3, 10, and 25 agents to measure natural redundancy patterns (total queries, unique queries, sub-expression overlap). Establish experiment tracking via Weights & Biases or MLflow with fixed random seeds and pinned library versions for reproducibility.

Phase 2 — Middleware Implementation (Month 2). Build the query interceptor using sqlglot for AST-level SQL normalisation and hashing. Implement exact-match caching (P1). Build the shared discovery board as a task-keyed SQLite store. Implement active result sharing (P2) with context-window-aware injection (concise summaries rather than full result sets). Run comparative experiments: P0 vs P1 vs P1+P2.

Phase 3 — Advanced Policies & Evaluation (Month 3). Implement a rule-based phase classifier to distinguish exploration from solution formulation (metadata/PRAGMA/LIMIT queries vs. full aggregations/joins), generalised for DuckDB-specific patterns. Wire P3 (discovery broadcasting) and P4 (staged coordination) into the middleware. Run the full evaluation matrix: 5 policies × 3 agent counts × 2+ LLMs. Begin thesis draft (introduction, background, system design chapters).

Phase 4 — Analysis & Writing (Month 4). Identify surprising findings and run targeted follow-up experiments. Produce key figures and tables. Report 95% bootstrap confidence intervals for all accuracy comparisons. Complete the evaluation, discussion, conclusion, and future-work chapters. Full revision pass with advisor review. Buffer time for unexpected issues.

4	Baselines complete	Redundancy report with query overlap analysis; baseline figures for thesis Chapter 2
8	Middleware P0–P2 operational	Working query interceptor and cache; cache elimination rate and speedup measurements
10	All policies (P0–P4) implemented	Phase-detection validation results; complete middleware system
12	Full evaluation matrix complete	Results tables and key figures; draft of Chapters 1–3
14	Analysis & follow-ups done	Evaluation and discussion chapters drafted; all statistical tests complete
16	Thesis submitted	Complete thesis document with all chapters, figures, and appendices

Skills: Python (asyncio, data analysis), SQL, basic distributed systems concepts (caching, coordination), experimental design and statistical analysis, academic writing.

Languages & Frameworks: Python 3.10+, SQL, JavaScript (optional for visualisation).

Libraries: sqlglot (SQL parsing/AST normalisation), DuckDB (Python API), asyncio (agent orchestration), pandas/numpy (analysis), matplotlib/seaborn (figures).

LLM APIs: OpenAI API (GPT-4o-mini as primary model), plus one secondary model — Claude 3.5 Haiku (API) or Qwen2.5-Coder-7B (local, if GPU available).

Infrastructure: DuckDB (in-process, no server needed), SQLite (discovery board), Weights & Biases or MLflow (experiment tracking), Git (version control).

Data: BIRD text-to-SQL benchmark dataset (publicly available).

Budget: LLM API credits for running experiments at scale. A hard budget cap should be agreed with the supervisor. Estimated at approximately \$200–\$500 depending on the number of configurations tested.

Wk 1–2	Infrastructure	Install DuckDB and load BIRD dataset. Build async Python orchestrator. Get single agent solving tasks end-to-end with full logging. Set up version control and experiment tracking.
Wk 3–4	Baselines	Run 3, 10, and 25 parallel agents. Log all queries. Build analysis scripts for total queries, unique queries, sub-expression overlap, and time-to-solution. Produce baseline redundancy report.
Wk 5–6	Cache layer	Use sqlglot for AST-level SQL normalisation and hashing. Implement exact-match caching (P1). Re-run workloads and measure cache elimination rate and speedup.
Wk 7–8	Sharing layer	Build shared discovery board (SQLite). Implement active result sharing (P2) with concise context injection. Run P0 vs P1 vs P1+P2 comparisons.
Wk 9–10	Advanced policies	Implement rule-based phase classifier. Wire P3 (discovery broadcasting) and P4 (staged coordination). Validate phase-detection accuracy.
Wk 11–12	Full evaluation	Run all policies × agent counts × LLMs. Collect seven metrics per configuration. Compute 95% confidence intervals. Begin thesis draft: Chapters 1–3.
Wk 13–14	Analysis	Targeted follow-up experiments. Produce key figures and tables. Draft evaluation and discussion chapters.
Wk 15–16	Thesis & buffer	Complete conclusion and future-work chapters. Full revision pass. Advisor review cycle. Buffer for unexpected issues. Submit final thesis.

This project definition has been undertaken in consultation with the supervisor. The feasibility of the project has been assessed and the project aims are clearly defined. Submission of this document implies that the specification has been discussed with the supervisor.